Ashish Nadadur Chakravarthi
anadadur@asu.edu

Q1. False, Bagging samples data points (bootstrap sampling), not features
Q2. False, Bagging uses sampling with replacement (bootstrap)
Q3. False, AdaBoost trains weak learners sequentially, not in parallel
Q4. True, Ensembles usually help, but not guaranteed.
Q5. True, Boosting builds sequentially; each learner corrects the previous one.
Q6. True, LTR models can be pointwise/pairwise/listwise and use many model types.
Q7. False, Agents interact with and change the environment through actions.
Q8. False, Q value is defined as the Expected discounted total future reward
Q9. False, RL uses experience from interaction, not supervised labeled data.
Q10. True, Margin is determined by support vectors only.
Q11. False, DQN is not a policy gradient method; it's a value-based deep Q-learning method.
Q12. True, DQN = Deep Q-Network (uses NN to approximate Q-function)

Q13. Samples that are correctly classified are 1, 2, 4, 5, 6, 7 and 8. because Adaboost decreases the weights of correctly classified samples so the next model focuses less on them.

The weights of misclassified samples are increased so that the next model focuses more on these "hard" examples

Sample 3 is misclassified.

Q14. The strategy is to increase the weights of misclassified data points and decrease the weights of correctly classified data points after each iteration.

Weights are increased so the next weak learner focuses more on them. This turns weak learners into a strong ensemble because each new model specifically targets the mistakes of the previous ones.

Q15. Correct : 1) Better performance : Combines the predictions of multiple base estimators

2) Generalized & robust models : Ensembles help reduce the risk of overfitting (variance) or underfitting (bias)

Q16. 1) Maximize the Margin between classes: SVM seeks to find the hyperplane that has the largest distance (margin) to the nearest training data points of any class

2) Minimize Classification Error: Aims to correctly separate the data points into their respective classes.

Q17. Support vectors are the specific data points (samples) that lie closest to the decision boundary (hyperplane) and determines the position and orientation of the SVM hyperplane.

- If you remove support vectors, the decision boundary changes
- If you remove non-support vectors, nothing changes.

Q18. 1) Linear Kernel: $K(x, x') = x^T x'$

Used when data is already linearly separable

2) Polynomial Kernel: $K(x, x') = (x^T x' + c)^d$

d = degree and c is constant

3) Radial Basis Function (RBF)/Gaussian Kernel

$$K(x, x') = \exp\left(-\frac{\|x - x'\|^2}{2\sigma^2}\right)$$

Q19. When the dataset contains noise, outliers, or is not perfectly linearly separable, we need soft margin SVMs.

Hard Margin SVM requires every single data point to be correctly classified and outside the margin. If there is a outlier this type of SVM will fail to find a solution.

Soft Margin SVM introduces slack variable that allow the model to ignore (misclassify) a few noisy points in exchange for a wider, more robust margin that generalizes better to new data.

Q20. We need kernel functions to handle non-linearly separable data. Many real-world datasets cannot be separated by a straight line. The kernel allows the SVM to implicitly map the input data into a higher-dimensional feature space where the classes do become linearly separable without the massive computational cost of actually transforming the data coordinates.

A kernel function measures the similarity between two data points (x and x') in that higher-dimensional space.

$$K(x, x') = \phi(x)^T \phi(x') \Rightarrow \text{dot product of 2 vectors in the high-dimensional space.}$$

Q21. The Polynomial Kernel and its Feature Space

Let our input vectors be $u=(u_1, u_2)$ and $v=(v_1, v_2)$.

$$K(u,v) = (1+u.v)^2$$

Here,

$$u.v = u_1v_1 + u_2v_2$$

So, $$K(u,v) = (1+u_1v_1+u_2v_2)^2$$

$$K(u,v) = 1 + u_1^2v_1^2 + u_2^2v_2^2 + 2u_1v_1 + 2u_2v_2 + 2u_1u_2v_1v_2$$

$$K(u,v) = (1)(1) + (\sqrt{2}u_1)(\sqrt{2}v_1) + (\sqrt{2}u_2)(\sqrt{2}v_2) + (u_1^2)(v_1^2) + (u_2^2)(v_2^2) + (\sqrt{2}u_1u_2)(\sqrt{2}v_1v_2)$$

This shows that the kernel computation is equivalent to a dot product in a 6-D feature space. The feature map $\phi(x)$ for a vector $x=(x_1, x_2)$ is:

$$\phi(x) = (1, \sqrt{2}x_1, \sqrt{2}x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2)$$

The feature space mentioned $(1, x_1, x_2, x_1^2, x_2^2, x_1x_2)$ is a simplified version created by the kernel. An SVM using these features is functionally equivalent to one using the kernel.

Therefore, an SVM using the polynomial kernel $K(u,v)=(1+u.v)^2$ is equivalent to a linear SVM operating in this higher-dimensional feature space.

We will show that the boundary of an ellipse becomes a linear separator (a hyperplane) in this new feature space.

- Ellipse equation: $c(x_1-a)^2+d(x_2-b)^2-1=0$

$$c(x_1^2-2ax_1+a^2)+d(x_2^2-2bx_2+b^2)-1=0$$

$$cx_1^2-2acx_1+a^2c+dx_2^2-2bdx_2+b^2d-1=0$$

- Grouping terms according to the components of our feature space: $(1, x_1, x_2, x_1^2, x_2^2, x_1x_2)$

$$(c)x_1^2+(d)x_2^2+(-2ac)x_1+(-2bd)x_2+(0)x_1x_2+(a^2c+b^2d-1)=0$$

This equation is a **linear combination** of the features in our new space

- We can express this as a dot product $w.z+b'=0$, which is the equation for a hyperplane. Let the feature vector be $z=(x_1, x_2, x_1^2, x_2^2, x_1x_2)$. Then the equation for the ellipse becomes a linear equation with:

Weight vector w: $w=(-2ac, -2bd, c, d, 0)$

Bias term b': $b'=a^2c+b^2d-1$

- This hyperplane divides the feature space into 2 regions:
  1) Points inside the ellipse satisfy $c(x_1-a)^2+d(x_2-b)^2-1<0$, which corresponds to $w.z+b'<0$
  2) Points outside the ellipse satisfy $c(x_1-a)^2+d(x_2-b)^2-1>0$, which corresponds to $w.z+b'>0$

Since the elliptic region can be separated from the rest of the plane by a single hyperplane in the feature space, we have shown that SVMs with a polynomial kernel of degree 2 can separate any elliptic region

Q22. Points: 2, 6, 10, 14, 18 If tie, merge left-most pair
Initial Centroids: 2, 6, 10, 14, 18

Step 1: Distances between adjacent centroids
$|6-2|=4$ $|10-6|=4$ $|14-10|=4$ $|18-14|=4$

⇒ Merged pair = (2, 6), centroid = $\frac{2+6}{2}=4$
(New cluster)

Step 2:

Current clusters: {2,6} (centroid 4), 10, 14, 18
Distances: $|10-4|=6$ $|14-10|=4$ $|18-14|=4$
Tie, Merge (10, 14)

⇒ New cluster: (10, 14), Centroid = 12

Step 3:

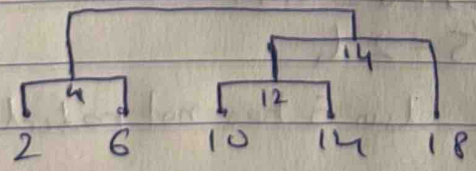
Current clusters: {2,6} (centroid 4), {10,14} (centroid 12), 18
Distances: $|12-4|=8$ $|18-12|=6$
Min, Merge (10,14) with (18)

⇒ New cluster: (10, 14, 18), Centroid = 14

	New Cluster	Centroid
Step 1	{2,6}	4
Step 2	{10,14}	12
Step 3	{10,14,18}	14

Final 2 clusters:
(2,6) and (10,14,18)

Dendogram:



Q23. b) Item-item collaborative filtering is the best choice because it identifies relationships between specific items that are frequently interacted with together (co-occurrence), making it ideal for detecting complementary pairs like "laptop and bag" or "pasta and sauce" regardless of the user's personal profile.

Q24. Baseline Estimate Formula,

$$\hat{r}_{ui} = \mu + b_u + b_i$$

where μ = Global mean rating

b_u = User bias

b_i = Item bias

Bob's avg rating higher than global

Pride & Prejudice avg rating is lower than global

Here, $b_u = +0.4$ and $b_i = -0.5$

$\therefore$ Estimate = $3.6 + 0.4 + (-0.5)$

Estimate = 3.5

Q25.

Estimate rating for item i based on rating for similar item

$$r_{xi} = \frac{\sum_{j \in N(i;x)} s_{ij} \cdot r_{xj}}{\sum s_{ij}}$$

Step 1: Mean ratings for each movie

Movie 1: $(1+3+5+5+4)/5 = 3.6$

Movie 3: $(2+4+1+2+3+4+3+5)/8 = 3.0$

" 4: $(2+4+5+4+2)/5 = 3.4$

" 5: $(4+3+4+2+2+5)/6 = 3.33$

" 6: $(1+3+3+2+4)/5 = 2.6$

Movie 2 is excluded because User 5 has not rated it.

Step 2: Pearson Correlation (Similarity)

S_{ij} between the target Movie 1 and the candidate neighbors (Movie 3, 4, 5, 6) using the centered cosine similarity formula. We compare users who rated both movies in the pair.

1. Similarity between Movie 1 and Movie 3 ($S_{1,3}$)
   - Common Users: 1, 9, 11
   - Centered Ratings (Rating − Mean):
     - User 1: M1 = (1 − 3.6) = −2.6, M3 = (2 − 3.0) = −1.0
     - User 9: M1 = (5 − 3.6) = 1.4, M3 = (4 − 3.0) = 1.0
     - User 11: M1 = (4 − 3.6) = 0.4, M3 = (5 − 3.0) = 2.0
   - Numerator (Dot Product): $(-2.6)(-1) + (1.4)(1) + (0.4)(2) = 4.8$
   - Denominator (Lengths):
     - $\|M1\| = \sqrt{(-2.6)^2 + (1.4)^2 + (0.4)^2} = \sqrt{8.88}$
     - $\|M3\| = \sqrt{(-1)^2 + (1)^2 + (2)^2} = \sqrt{6}$
   - $S_{13} = \frac{4.8}{(\sqrt{8.88} \cdot \sqrt{6})} \approx 0.658$

2. Similarity between Movie 1 and Movie 6 $(S_{1,6}) = \frac{4.48}{2.698 \times 2.163}$
   $S_{1,6} = 0.767$

3. $S_{1,4}$ & $S_{1,5}$ have negative correlations, so they are less similar than Movie 3 and 6.

Step 3: We need a neighbor set size $|N(i;x)| = 2$. So, we choose Movie 3 ($S = 0.658$) and Movie 6 ($S = 0.533$)

Step 4 Rating

$$r_{1,5} = \frac{S_{1,3} \cdot r_{3,5} + S_{1,6} \cdot r_{6,5}}{S_{1,3} + S_{1,6}}$$

Here, $S_{1,3} \approx 0.658$
$S_{1,6} \approx 0.767$
$r_{3,5}$ (User 5's rating for Movie 3) = 2
$r_{6,5}$ (" " " " 6) = 3

$$r_{1,5} = \frac{(0.658 \times 2) + (0.767 \times 3)}{0.658 + 0.767} = \frac{3.617}{1.425} \approx 2.538 \approx 2.54$$

The estimated rating for User 5 on Movie 1 is around 2.54.

Q26. user feature (u), item feature (i), sparse matrix of ratings (r)

1) Creation of feature vectors and corresponding labels
   - Create a combined feature vector for every interaction. For a specific user(u) and Item (i), construct an input vector x_{ui} by concatenating the user features and item features.
     $x_{ui} = [\text{features}_u, \text{features}_i]$
   - Use pairwise approach to construct pairs of items for a user. If user u rated item i higher than item j, we create a training instance (x_{ui}, x_{uj}) with the target relationship $i > j$.

2) Architecture: Deep Neural N/w (Multi-Layer Perceptron)
I/p layer: Takes the concatenated vector x_{ui} (User Features + Item Features)
Hidden layers: Several non-linear layers (e.g. ReLU activation) to learn complex interactions between user preferences and item attributes
O/p layer: A single node outputting a scalar score $s_{ui} = f(x_{ui}; \theta)$

3) Pairwise Logistic Loss (RankNet style) Loss

$$L(\theta) = \sum_{(u,i,j) \in D} -\ln(\sigma(s_{ui} - s_{uj}))$$

σ is sigmoid function, and the sum is over all pairs where item i is rated higher than item j by user u.

4)

4) 1. Candidate generation: Select a set of candidate items for the target user
2. Feature Construction: For every candidate item, construct the feature vector x_{ui} using the target user's features
3. Scoring: Feed these vectors into the trained model to predict a relevance score s for each item.
4. Ranking: Sort the items in descending order based on their predicted scores
5. Output: Return the top K items as the personalized recommendation list.

Q27. 1) Pointwise: Treats ranking as a standard regression or classification problem. It predicts the exact relevance score for a single item.
2) Pairwise: Learn which of two items should rank higher
3) Listwise: Treats the entire list of items as a single training instance. Optimizes a ranking metric directly over the whole list.

Q28. 1) RankSVM: ~~Hinge loss~~ Hinge Loss (Pairwise hinge loss)
2) RankBoost: Exponential loss
3) RankNet: Logistic loss (Cross-Entropy / Log Loss / Pairwise logistic loss)

Q29. 1) The ranking error indicator function is a 0-1 Loss function. For a pair of items where item i should be ranked higher than item j, let $z = f(x_i) - f(x_j)$ be the difference in their predicted scores.

$$I(z<0) = \begin{cases} 1 & \text{if } z<0 \text{ (Error: Wrong Order)} \\ 0 & \text{if } z \geq 0 \text{ (Correct order)} \end{cases}$$

2) It represents the ideal objective for the ranking task. We want to minimize the total count of misordered pairs (inversions). But this function is non-differentiable and cannot be optimized directly.

3) RankSVM - Hinge loss
- Uses a convex upper bound on the indicator
- If the pair is misordered, hinge loss is +ve.
- As model correctly increases $s_i - s_j$, the loss monotonically decreases toward zero.

RankBoost - Exponential loss
- Uses exponential loss as a smooth, differentiable upper bound
- Misordered pairs give large loss, correct pairs give small loss
- As ranking improves, the loss monotonically decreases.

RankNet - Cross Entropy / Logistic Loss
- Models pairwise preference probability and uses cross-entropy.
- Misordered pairs produce high loss; correctly ordered pairs produce small loss
- As the difference moves in correct direction, the loss monotonically decreases.

All 3 methods minimizes ranking mistakes by using surrogate losses that upper-bound the ranking error indicator function. Because these surrogate losses are designed to:

1) penalize misranked pairs more heavily
2) reduce smoothly when ordering improves

the loss functions all monotonically decrease as the ranking gets better

Q30. Model 2 is better because it produces the correct relative ordering of items, not predict the exact numerical scores (regression).

1) Golden Standard (Actual) Order : Item A (0.9) > Item B (0.85) > Item C (0.8)
Correct Rank : A, B, C

2) Model 1 :
B (0.86) > A (0.85) > C (0.81)
Rank : B, A, C

3) Model 2 :
A (0.2) > B (0.15) > C (0.1)
Rank : A, B, C

Even though Model 1 scores are numerically closer to the ground truth (lower MSE), Model 2 is better ranking model because it results in the correct list order.

Q31.1) Bipartite Ranking Error

Relevant Set (S_+) : D3, D4, D5, D6 → 4 +ve

Non-Relevant Set (S_-) : D1, D2, D7, D8 → 4 -ve

Total pairs = $|S_+| \times |S_-| = 4 \times 4 = 16$

Disordered pairs means Non-relevant document has a higher score than a relevant document. We need to check the non-relevant items (D1, D2, D7, D8) against the scores of the relevant items (D3, D4, D5, D6).

Relevant scores : D3 (0.73), D4 (0.64), D5 (0.55), D6 (0.46)

D1

0.91 > all Relevant scores

∴ 4 Disordered pairs

D2

0.82 > all Relevant scores

∴ 4 Disordered pairs

D7

0.37 < all Relevant scores

∴ 0 disordered pairs

D8

0.28 < all Relevant scores

∴ 0 disordered pairs

Total disordered pairs = 4+4+0+0 = 8

Bipartite Ranking Error = $\frac{8}{16} = 0.5$

2) Binary Classification Error = $\frac{2}{8} = 0.25$

Only document D1 and D2 are incorrectly classified.

Q32. The agent should take Action a_1.
In Q-learning, when an agent needs to decide the best action to take in a specific state, it looks for the action that has the highest Q-value in that state's column. This represents the highest expected cumulative reward.

$$Q(s_2, a_1) > Q(s_2, a_2) > Q(s_2, a_3)$$
$$2.40 \qquad 0.7 \qquad -2.80$$

$\therefore$ $Q(s_2, a_1)$ is highest

Q33.

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma \max_{a_{t+1}} Q(s_{t+1}, a_{t+1})$$

Action taken : a_1
Current state (s_t) : s_2
Next State (s_{t+1}) : s_4
Reward (r) : 0.6
Discount factor (γ) : 0.9

Max future Q-value at s_4 $\quad Q(s_4, a_2) > Q(s_4, a_1) > Q(s_4, a_3)$
$\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\quad 1.50 \qquad 1.47 \qquad 0.80$

$$\text{Max } Q = 1.5$$
$$Q(s_2, a_1) = 0.6 + 0.9 \times (1.50)$$
$$= 1.95$$

	s1	s2	s3	s4
a1	1.20	1.95	2.50	1.47
a2	0.33	0.70	1.31	1.50
a3	-0.90	-2.80	1.00	0.80

Q34.

1) a) Policy Evaluation: Calculating the value function (expected rewards) for the current policy using the Bellman equation.

b) Policy Improvement: Updating the policy to be "greedy" wrt the estimated value function.

2) a) Sampling / Interaction: Running the current policy to generate trajectories (sequences of states, actions, and rewards)

b) Parameter Update: Calculating the gradient of the expected reward wrt the policy parameters and updating the parameters (usually via gradient ascent) to increase the probability of high-reward actions.

3) We should use Value Learning (or Value-based methods like Deep Q-Network (DQN) or Q-Learning)

They work efficiently with discrete actions because it is computationally easy to calculate the maximum value over a finite set of actions.

4) Weaknesses include:

1) Not directly optimized for stochastic or continuous actions (limited to discrete).
2) Overestimation bias in Q-values
3) Hard to learn good policies in very large or continuous state-action spaces due to the need to store or approximate $Q(s,a)$ for each action
4) Instability and divergence issues with function approximation.